dcs/ipl/

CSDC102 |

Intermediate Programming

Final Programming Project |

Road Crossing Challenge

Duration: 3 Weeks
Activity Type: Group Project (5 members per group)
Language: C++
Submission: Source code (.cpp), Video Gameplay (.mp4), and documentation (PDF)

Instructions |

Complete all sections. Write your code clearly and test your logic. This worksheet will help you prepare for strings and vectors next week!

Disclaimer ⚠️ |

Please attempt to solve the problems using your own understanding and capability before seeking answers from the internet, classmates, or AI tools such as ChatGPT. Struggling through the problem is part of the learning process and will help strengthen your programming skills.

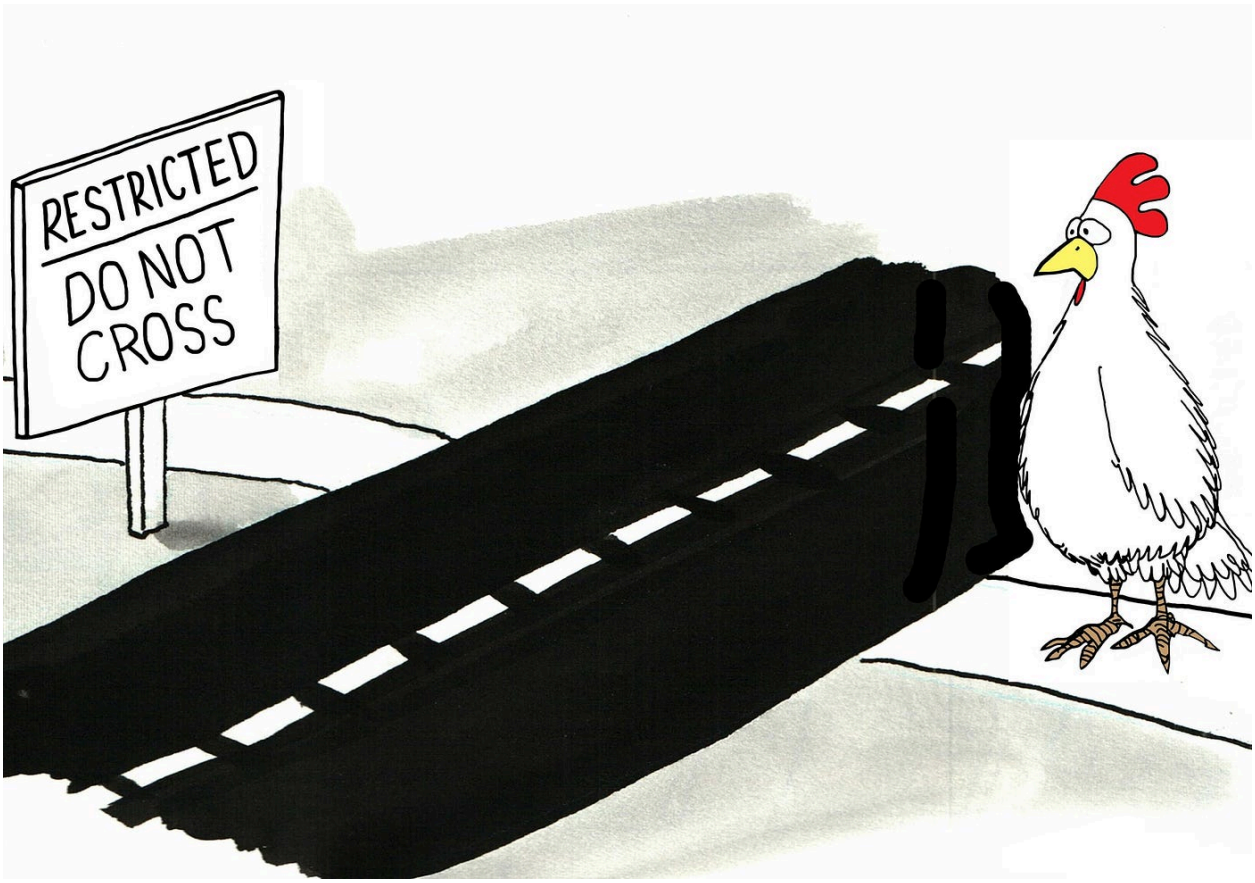
If you do consult external resources, you must properly acknowledge them by listing all sources at the end of your answers.

General Instructions for All Exercises |

1. **Read the Problem Carefully:** Understand the scenario and what the program is expected to accomplish.
2. **Plan Your Approach:** Identify which data structures (arrays, matrices) and functions you will use in your C++ program.
3. **Implement in C++:** Write the program according to the specifications using proper syntax, functions, loops, and array/matrix operations.
4. **Produce the Expected Output:** Ensure your program outputs match the required format exactly.
5. **Test Your Program:** Run multiple test cases to verify correct behavior, including edge cases (e.g., empty arrays, maximum size).
6. **Maintain Good Coding Practices:** Use meaningful variable names, proper indentation, and comments where needed.

Important Guidelines |

1. **No LLMs or AI Assistance:** Do not use AI tools (e.g., ChatGPT, Bard) to generate solutions for this laboratory exercise. All work must be your own.
2. **Confidentiality of Exercises:** Do not share or leak the laboratory exercises, problem statements, or solutions to students in other sections.
3. **Academic Honesty:** Collaborate only as instructed by the instructor. Any form of plagiarism or unauthorized sharing will be dealt with according to university policies.



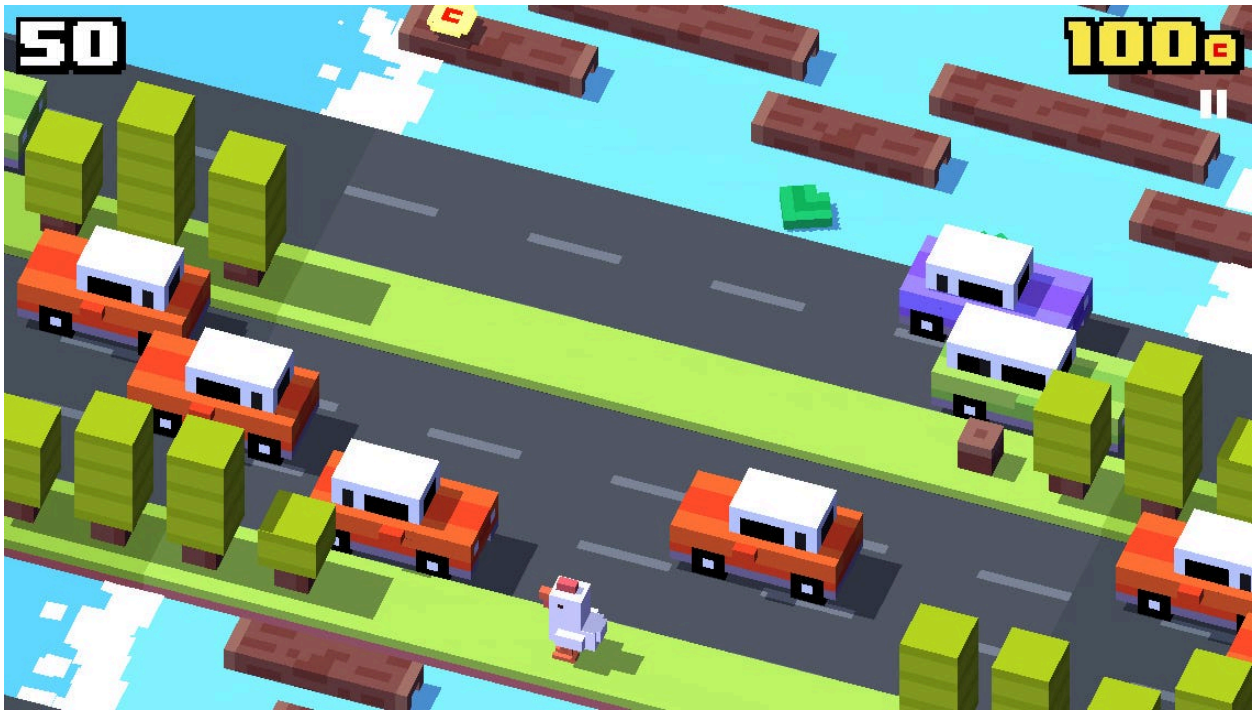
"Why did the chicken cross the road? …
It wasn't crossing, it was just updating its head->next."

Project Overview |
Designing Your Own “Crossy Road” Game in Terminal Using C++

You will develop a terminal-based C++ game where a player must cross multiple lanes of obstacles. This enhanced version introduces multi-terrain gameplay, persistent scoring, and improved user interaction.

The goal is to reach the top safely while navigating both road traffic and river hazards.

Game Overview | The Game: Crossy Road



Crossy Road by Hipster Whale and yodo1

Crossy Road is a popular arcade-style mobile game released in 2014 by the Australian indie studio Hipster Whale. It takes a simple idea of chicken or a livestock crossing roads and turns it into a fast-paced, endless challenge.

Core Concept |
At its heart, Crossy Road answers the classic joke: “Why did the chicken cross the road?” But instead of just crossing once, the goal is to keep going as far as possible without dying.

Section 1 | Main Requirements

To ensure proper implementation and good programming practices, the following requirements must be followed:

- Except for constant values, your program must not use global variables.
- A linked list must be used to represent each lane of the road, allowing dynamic and efficient updates of game elements.
- Your program must be free from memory leaks and dangling pointers. Proper allocation and deallocation of memory is required.
- Implement a leaderboard system using file handling (fstream) to store and retrieve player scores.
- The game must include multi-condition logic to handle different game states (e.g., movement, collision, win/lose conditions).
- Proper use of modular and structured programming is encouraged for readability and maintainability.

Required Data Structure

Use the following structure for your linked list implementation:

```
C/C++
struct Node{
    string data;
    Node* next;
};
typedef Node* NodePtr;
```

What Makes This Game Special

Five key features distinguish this from a basic obstacle game:

- Randomized Terrain: Every game generates a new mix of Road and River zones in a fixed pattern (5 Road lanes, then 1 Buffer, then 2 River lanes, then 1 Buffer, repeating).
- Road Zone: Moving trucks (##### blocks) you must dodge.
- River Zone: Flowing logs (====) you must hop onto and ride across.
- Buffer Zone: A guaranteed safe lane with no obstacles between each zone pair.
- Leaderboard: Scores saved to leaderboard.txt and displayed after every game.

Section 2 | The Game Field

The game field is the main playable area where all game actions take place. It defines the space, boundaries, and layout that guide how the player interacts with the environment. In an online or digital game setting, the game field is usually represented visually using grids, tiles, or character-based maps that simulate real-world or fictional spaces. It is within this structured area that elements such as the player, obstacles, objectives, and hazards are placed and updated in real time, creating the core experience of gameplay.

1. **Dimensions**
    - You will create a road of **40 characters in width** and **20 lines in height** (lanes).
    - Outside the road, there is a title, two barriers, and a line showing lives/score.
    - **Total dimensions: 42 characters × 22 lines**
        - Game Width: 42 characters (40 playfield + 2 border walls '|')
        - Game Height: 22 lines total (see Section 3 for full breakdown)

2. **Characters**
    - **Player: 'P'** represents the player
        - Starting Position: Column 20, Row 19 (bottom-centre)
    - **Truck: "#####"** represents moving vehicles
        - These are connected blocks of 5 '#' characters
        - Touching this kills the player
    - **Log: "===="**
        - These are 4 connected blocks '=' characters
        - This is a safe space in the river zone.
    - **Road: "."** represents safe spaces on the road
        - No obstacles, always safe
    - **Water: "~" (tilde)**
        - Touching this drowns the player
    - **Finish Line: "="** represents the finish line (top of screen)

3. **Player Movement**
    - The player can move **UP, DOWN, LEFT, RIGHT** using arrow keys
    - Player starts at position (20, 20) - middle bottom of the screen

Section 3 | The Randomized Terrain System

This is the most important structural concept in the project. Instead of hard-coding a fixed map, the game generates terrain in repeating blocks. Each block follows a strict pattern: 5 Road lanes, then 1 Buffer lane, then 2 River lanes, then 1 Buffer lane. This block repeats going up the screen until the finish line is reached.

3.1 | The Zone Block Pattern

Think of the 20 playable lanes as being divided into groups from bottom (where the player starts) to top (the finish line). One complete group looks like this:

ROAD (5 lanes)	BUFFER (1 lane)	RIVER (2 lanes)	BUFFER (1 lane)	ROAD (5 lanes)	...

One Complete Terrain Block = 9 lanes total
5 lanes: ROAD ZONE (trucks move left/right)
1 lane: BUFFER ZONE (all dots, no obstacles)
2 lanes: RIVER ZONE (logs drift left/right)
1 lane: BUFFER ZONE (all dots, no obstacles)
This group of 9 lanes repeats from the bottom of the screen upward.

📝 **Note:**
The word 'randomized' here means the content of each lane is randomly generated (how many trucks, where they start), not that the zone type changes each run. The pattern of zone types—Road, Buffer, River, Buffer—is always the same. This guarantees the game is always crossable.

3.2 | Full 20-Lane Layout

With 20 playable lanes and a 9-lane block, two full blocks fit with 2 spare lanes at the bottom for the player start area. Here is the exact top-to-bottom assignment:

Row Index	Zone Type
Row 0	FINISH LINE: '|==========|'
Row 1	ROAD — truck lane (lane number 1, moves RIGHT)
Row 2	ROAD — truck lane (lane number 2, moves LEFT)
Row 3	ROAD — truck lane (lane number 3, moves RIGHT)
Row 4	ROAD — truck lane (lane number 4, moves LEFT)
Row 5	ROAD — truck lane (lane number 5, moves RIGHT)

Row 6	BUFFER — safe lane, all dots '.'
Row 7	RIVER — log lane (lane number 1, logs move RIGHT)
Row 8	RIVER — log lane (lane number 2, logs move LEFT)
Row 9	BUFFER — safe lane, all dots '.'
Row 10	ROAD — truck lane (lane number 6, moves LEFT)
Row 11	ROAD — truck lane (lane number 7, moves RIGHT)
Row 12	ROAD — truck lane (lane number 8, moves LEFT)
Row 13	ROAD — truck lane (lane number 9, moves RIGHT)
Row 14	ROAD — truck lane (lane number 10, moves LEFT)
Row 15	BUFFER — safe lane, all dots '.'
Row 16	RIVER — log lane (lane number 3, logs move RIGHT)
Row 17	RIVER — log lane (lane number 4, logs move LEFT)
Row 18	BUFFER — safe lane, all dots '.'
Row 19	PLAYER START — empty lane '| |'

💡 **Tip:**
The player starts at Row 19. Moving UP decreases playerY. The finish line is Row 0. The player wins when playerY reaches 0.

3.2 | Storing Zone Types in an Array
To make collision detection and zone-specific logic easy, store the type of each lane in a parallel array. This lets you check what kind of zone the player is in without re-reading the lane string.

3.3 | How "Randomized" Content Works Per Lane
Even though the zone pattern is fixed, the content inside each lane is randomly generated every time buildRoad() is called (i.e., at game start and after each crossing). Two lanes of the same zone type will look different from each other because truck positions and log positions are chosen with rand().

ROAD lane (randomized trucks)	RIVER lane (randomized logs)
| ######...######...###### |	| ~~~~====~~~~~~~~~~====~~~~~~~ |
| ....######........##### |	| ====~~~~~~~~====~~~~~~~~~~==== |
| ######......######........ |	| ~~====~~~~~~~~~~====~~~~~~~~~ |
(same zone type, different random content each game)	(same zone type, different random content each game)

Section 3.a | The Full Game Screen

Here is what a typical screen looks like with the terrain pattern applied. Color is for illustration only (but you can implement this using other colors as well). In the terminal, everything appears in the default console font.

COMPLETE GAME SCREEN (42 chars wide x 22 lines tall)

```
---------- Road Crossing Challenge ----------
Player: Ian | Lives: 3 | Crossings: 1
|=========================================|   <- Row 0  FINISH
| #####...#####.........#####.......... |   <- Row 1  ROAD
| ......#####.......#####........#####.. |   <- Row 2  ROAD
| #####.......#####.......#####......... |   <- Row 3  ROAD
| ......#####.......#####........#####.. |   <- Row 4  ROAD
| #####.......#####.......#####......... |   <- Row 5  ROAD
|.........................................|   <- Row 6  BUFFER (safe!)
| ~~~~====~~~~~~~~~~~~~~~~~~~~~~~~====~~~~~~~ |   <- Row 7  RIVER
| ====~~~~~~~~~~~~~~~~~~~~~~~~~~====~~~~~~~~~~ |   <- Row 8  RIVER
|.........................................|   <- Row 9  BUFFER (safe!)
| #####...#####.........#####.......... |   <- Row 10 ROAD
| ......#####.......#####........#####.. |   <- Row 11 ROAD
| #####.......#####.......#####......... |   <- Row 12 ROAD
| ......#####.......#####........#####.. |   <- Row 13 ROAD
| #####.......#####.......#####......... |   <- Row 14 ROAD
|.........................................|   <- Row 15 BUFFER (safe!)
| ~~~~====~~~~~~~~~~~~~~~~~~~~~~~~====~~~~~~~ |   <- Row 16 RIVER
| ====~~~~~~~~~~~~~~~~~~~~~~~~~~====~~~~~~~~~~ |   <- Row 17 RIVER
|.........................................|   <- Row 18 BUFFER (safe!)
|                   P                     |   <- Row 19 START
Lives: 3 | Crossings: 0
```

📝 **Note:**
The status line 'Lives: X | Crossings: Y' is printed AFTER the linked list, not stored in the list. It is just a cout statement at the end of displayRoad().

Section 4 | Zones

4.1 | Buffer Zone: The Safe Lane

A buffer zone is a lane with no obstacles at all. It exists at row 6, 9, 15, and 18. The player can stand on a buffer lane indefinitely without losing a life. Think of it as the sidewalk between zones.

Generating a Buffer Lane

A buffer lane is the simplest lane to generate—it is just a string full of dots surrounded by border characters.

Buffer Lanes Never Move

When shiftObstacles() runs every game loop, it must skip buffer lanes. Use zoneMap to check the lane type before shifting.

> 💡 **Tip:**
> Buffer lanes serve two gameplay purposes: they give the player a moment to breathe and plan the next move, and they act as a clear visual separator between a dangerous Road zone and a dangerous River zone.

Collision Logic for Buffer Zones

No special collision check is needed for buffer zones. Since checkCollision() only looks for '#' and checkDrowned() only looks for '~', a lane full of '.' characters will never trigger either condition.

4.2 | Road Zone: Look Out for Trucks

The road zone contains 5 lanes per block. Each lane has 2 to 3 trucks, each truck being exactly 5 consecutive '#' characters. Odd-numbered road lanes move RIGHT; even-numbered road lanes move LEFT.

4.3 | River Zone: Logs and Water

The river zone is the trickiest part to understand. The key rule is the opposite of the road zone: in the river, the log ('=') is safe and the water ('~') is deadly.

River Zone Core Rule

```
Road Zone:     '#' = DANGER  |  '.' = SAFE
River Zone:    '~' = DANGER  |  '=' = SAFE
```

The player must jump onto a log and ride it. If they fall into '~' — even from a log moving off-screen — they lose a life.

Shifting Strings (Left and Right)
Both road and river lanes use the same two shift functions. The wrap-around is critical—characters that fall off one edge must reappear at the other.

> ⚠️ **Warning:**
> The border characters '|' at positions 0 and 41 are part of the string stored in the node. When you shift the inner 40 characters, shift only positions 1 through 40, not the full 42-character string. Alternatively, strip the borders before shifting and re-add them after.

Player Riding a Log
Each game loop, if the player is in a river lane, they must move with the log they are standing on. This happens BEFORE the keyboard input is applied so the player's position is always up-to-date.

Section 5 | Title Screen & Player Name Entry

Every good game starts with a title screen. This is the first thing the player sees. It should display the game name, a prompt to press Enter, and then ask for the player's name.

5.1 | What the Title Screen Shows (Suggested)

TITLE SCREEN

```
 ____                  _     ____                   _
|  _ \ ___   __ _  __| |  / ___|_ __ ___  ___ ___(_)_ __   __ _
| |_) / _ \ / _` |/ _` | | |   | '__/ _ \/ __/ __| | '_ \ / _` |
|  _ < (_) | (_| | (_| | | |___| | | (_) \__ \__ \ | | | | (_| |
|_| \_\___/ \__,_|\__,_|  \____|_|  \___/|___/___/_|_| |_|\__, |
                  C h a l l e n g e                       |___/

 Written for CSDC102 | Language: C++

 HOW TO PLAY:
 - Move with Arrow Keys
 - Dodge trucks (#####) in the ROAD ZONE
 - Hop on logs (====) in the RIVER ZONE
 - Reach the finish line 5 times to win!

 Press ENTER to start...
```

5.2 | Implementing the Title Screen

The title screen is a simple function that clears the screen, prints the banner, and waits for the player to press Enter. After that, ask for their name.

5.3 | Difficulty Selection (Optional)

After entering their name, ask the player to choose a difficulty level. This changes the game speed (the Sleep delay) and potentially obstacle density.

Section 6 | Leaderboard System (File I/O)

The leaderboard saves each player's name and score to a file called leaderboard.txt. Every time someone finishes a game (win or lose), their result is appended. The leaderboard is displayed sorted by score.

6.1 | The Leaderboard File Format

Each line in leaderboard.txt stores one entry: the player's name and their score, separated by a space.

leaderboard.txt (example file contents)

```
Ian 5
Miguel 3
Kurt 5
Sophia 1
Jerome 2
```

6.2 | The Leaderboard Struct

Add a second struct to store leaderboard entries. This goes at the top of your file alongside the Node struct.

```
struct LeaderboardEntry {
	string name;
	int score;
};
```

6.3 | Saving a Score

Call a function at the end of the game, passing in the player's name and their final number of crossings.

6.4 | Loading and Displaying the Leaderboard

Read all entries from the file, sort them by score (highest first), and display the top 5. You can use a simple array or vector to hold the entries.

> 📝 **Note:**
> You must #include <fstream> at the top of your .cpp file to use ifstream and ofstream. No global variables are used here since the file is opened and closed within the function.

Section 7 | Linked List Structure

The entire game board is stored as a linked list where each Node represents one horizontal lane. The head of the list is the TOP of the screen (finish line), and the tail is the BOTTOM (player start).

7.1 | Required Struct (from Project Spec)

```
struct Node {
	string data;
	Node* next;
};
typedef Node* NodePtr;
```

7.2 | Building the Initial List

At the start of the game, create 21 nodes: 1 finish line + 5 river lanes + 1 safe buffer + 9 road lanes + 1 safe buffer + 1 player start row.

7.3 | Navigating to a Specific Lane

Since this is a singly-linked list, you must traverse from head to reach any lane by index. Write a helper function for traversing the list.

7.4 | Displaying the Road (with Player)

Walk the list from head to tail. At the player's row, overlay 'P' at playerX before printing.

7.4 | Memory Management (No Memory Leaks!)

At the end of the game, delete every node in the linked list. Walk from head to tail, deleting each node.

> 📝 **Note:**
> Always set head = NULL after freeing so no dangling pointer remains. Call freeList(head) before the program exits or before starting a new game.

Section 8 | All Functions to Implement

Here is every function you need, listed in the order you should implement and test them.

showTitleScreen()	Displays game title and waits for Enter
getPlayerName()	Asks player to type their name, returns string
chooseDifficulty()	Asks 1 or 2, returns int (difficulty level)
shiftLeft(lane)	Shifts a string one character to the left (wrap-around)
shiftRight(lane)	Shifts a string one character to the right (wrap-around)
generateRoadLane(laneNum, moveCounter)	Returns a string with 2-3 trucks (##### each)
generateRiverLane(laneNum, moveCounter)	Returns a string with 2-3 logs (==== each) in ~~~ water
getLane(head, index)	Returns pointer to the node at position index
buildRoad(moveCounter)	Creates the full 21-node linked list, returns head
shiftObstacles(head, moveCounter)	Updates every lane's string (shift trucks and logs)
displayRoad(head, playerX, playerY, ...)	Prints every lane, overlays 'P' at player position
checkCollision(head, playerX, playerY)	Returns true if player hits '#' in road zone
checkDrowned(head, playerX, playerY)	Returns true if player is on '~' in river zone
updatePlayerWithLog(playerX, playerY, ...)	Moves player with the log they are standing on
keyboardInput(input, playerX, playerY, ...)	Reads arrow key and updates position
gameStatus(lives, crossings)	Prints lives and crossings, end message if needed
saveScore(name, score)	Appends name and score to leaderboard.txt
showLeaderboard()	Reads leaderboard.txt, sorts, displays top 5
freeList(head)	Deletes all nodes in the linked list (no memory leaks)

Section 9 | Code Organization (Header Files)

To improve code readability and modularity, you are encouraged to separate your program into multiple files using header files.

Suggested Structure:

- **main.cpp** → contains the main game loop
- **game.h / game.cpp** → core game logic (movement, collisions, updates)
- **lane.h / lane.cpp** → lane generation and obstacle/log behavior
- **leaderboard.h / leaderboard.cpp** → file handling for scores

Example Header File (game.h):

```
C/C++
#ifndef GAME_H #define GAME_H

void showTitleScreen(); void displayRoad(...); bool
checkCollision(...);

#endif
```

📝 **Note:**
While not strictly required, proper file separation will earn better code quality and organization marks.

- Use #include "filename.h" to include your headers
- Avoid putting function definitions inside header files
- Ensure proper use of include guards (#ifndef, #define, #endif)

Section 10 | Reference: Falling Money

Your instructor wrote a similar game in Programming 2 (CSDC102) called 'Falling Money'. The Road Crossing Challenge builds on those same foundations. Here is how the two games map to each other:

Falling Money Concept	Road Crossing Equivalent
Single linked list of rows	Same — NodePtr head, each node = one lane
randomGen() for symbols	generateRoadLane() / generateRiverLane() / generateBufferLane()
border[kbpos] == '$' → score++	checkCollision(): lane[playerX]=='#' → lives--
border[kbpos] == '.' → score--	checkDrowned(): lane[playerX]=='~' → lives--
kbpos (left/right only)	playerX + playerY (now 2D movement)
score starts 50, win at 100	lives start at 3, win = 5 crossings
displayList() prints the list	displayRoad() does the same + 'P' overlay
Sleep(100) for speed	Sleep(gameSpeed) — changes with difficulty
No file saving	NEW: saveScore() + showLeaderboard() + fstream
No title screen	NEW: showTitleScreen() + getPlayerName()
Fixed lane types	NEW: zoneMap[] defines Road/River/Buffer per row
No safe zones	NEW: BUFFER lanes guarantee a rest area

📝 **Note:**
The biggest new concept compared to Falling Money is the zoneMap array. It decouples what a lane looks like (its string data) from what kind of danger it contains (its zone type). This makes the main loop's logic much cleaner than checking the lane string characters directly.

Section 11 | Sample Screens

Your instructor wrote a similar game in Programming 2 (CSDC102) called 'Falling Money'. The Road Crossing Challenge builds on those same foundations. Here is how the two games map to each other:

11.1 | Title Screen

TITLE SCREEN

```
  ==========================================
   ROAD CROSSING CHALLENGE
   A Crossy Road Terminal Game in C++
  ==========================================

  HOW TO PLAY:
  Arrow Keys: UP / DOWN / LEFT / RIGHT
  Dodge trucks (#####) in the ROAD zones.
  Ride logs (====) in the RIVER zones.
  Rest on buffer lanes (....) – they are safe!
  Reach the top 5 times to WIN!

  Press ENTER to continue...
```

11.2 | Difficulty Selection

DIFFICULTY SELECTION

```
  SELECT DIFFICULTY
  1. Easy  – speed: 180ms, 2 trucks/logs per lane
  2. Hard  – speed: 100ms, 3 trucks/logs per lane

  Enter choice (1 or 2): _
```

11.3 | Game Start (Easy Mode)

GAME START

```
---------- Road Crossing Challenge ----------
Player: Ian | Lives: 3 | Crossings: 1
|======================================|   <- Row 0  FINISH
| #####...#####.........#####.......... |   <- Row 1  ROAD
```

```
| ......#####.......#####.........#####.. |   <- Row 2  ROAD
| #####.......#####.......#####.......... |   <- Row 3  ROAD
| ......#####.......#####.........#####.. |   <- Row 4  ROAD
| #####.......#####.......#####.......... |   <- Row 5  ROAD
|........................................|   <- Row 6  BUFFER (safe!)
| ~~~~====~~~~~~~~~~~~~~~~~~~~~~~~====~~~~~~~ |   <- Row 7  RIVER
| ====~~~~~~~~~~~~~~~~~~~~~~~~~~====~~~~~~~~~~ |   <- Row 8  RIVER
|........................................|   <- Row 9  BUFFER (safe!)
| ######...#####..........#####.......... |   <- Row 10 ROAD
| ......#####.......#####.........#####.. |   <- Row 11 ROAD
| #####.......#####.......#####.......... |   <- Row 12 ROAD
| ......#####.......#####.........#####.. |   <- Row 13 ROAD
| #####.......#####.......#####.......... |   <- Row 14 ROAD
|........................................|   <- Row 15 BUFFER (safe!)
| ~~~~====~~~~~~~~~~~~~~~~~~~~~~~~====~~~~~~~ |   <- Row 16 RIVER
| ====~~~~~~~~~~~~~~~~~~~~~~~~~~====~~~~~~~~~~ |   <- Row 17 RIVER
|........................................|   <- Row 18 BUFFER (safe!)
|                   P                    |   <- Row 19 START
Lives: 3 | Crossings: 0
```

11.3 | Player Resting on a Buffer Lane

PLAYER ON BUFFER (row 9) — completely safe

```
---------- Road Crossing Challenge ----------
Player: Ian | Lives: 3 | Crossings: 1
|========================================|
| ######...#####..........#####.......... |
| ......#####.......#####.........#####.. |
| #####.......#####.......#####.......... |
| ......#####.......#####.........#####.. |
| #####.......#####.......#####.......... |
|........................................|
| ~~~~====~~~~~~~~~~~~~~~~~~~~~~~~====~~~~~~~ |
| ====~~~~~~~~~~~~~~~~~~~~~~~~~~====~~~~~~~~~~ |
|..............P.........................|   <- Player safe on buffer!
| ######...#####..........#####.......... |
| ......#####.......#####.........#####.. |
| #####.......#####.......#####.......... |
| ......#####.......#####.........#####.. |
| #####.......#####.......#####.......... |
```

```
|........................................|
| ~~~~====~~~~~~~~~~~~~~~~~~~~====~~~~~~ |
| ====~~~~~~~~~~~~~~~~~~~~====~~~~~~~~~~ |
|........................................|
|                                        |
Lives: 3 | Crossings: 0
```

11.3 | Player Riding a Log

PLAYER ON LOG (row 7) — safe while on '='

```
---------- Road Crossing Challenge ----------
Player: Ian | Lives: 3 | Crossings: 1
|========================================|
| #####...#####..........#####.......... |
| ......#####.......#####........#####.. |
| #####.......#####.......#####......... |
| ......#####.......#####........#####.. |
| #####.......#####.......#####......... |
|........................................|
| ~~~~==P=~~~~~~~~~~~~~~~~~~~~====~~~~~~ |   <- Player riding log
| ====~~~~~~~~~~~~~~~~~~~~====~~~~~~~~~~ |
|........................................|
| #####...#####..........#####.......... |
| ......#####.......#####........#####.. |
| #####.......#####.......#####......... |
| ......#####.......#####........#####.. |
| #####.......#####.......#####......... |
|........................................|
| ~~~~====~~~~~~~~~~~~~~~~~~~~====~~~~~~ |
| ====~~~~~~~~~~~~~~~~~~~~====~~~~~~~~~~ |
|........................................|
|                                        |
Lives: 3 | Crossings: 0
```

11.3 | End Game + Leaderboard

GAME OVER SCREEN

```
Game Over, Ian! You were hit too many times.

===== LEADERBOARD (Top 5) =====
1. Jose Rizal    – 5 crossing(s)
2. Ian Peter     – 3 crossing(s)
3. Miguel Damien – 3 crossing(s)
4. Jerome NOCS   – 2 crossing(s)
5. Si Cruch ko   – 1 crossing(s)
================================
```

Section 12 | Deliverables and Grading Criteria

Deliverable	Description
.cpp file	Complete C++ source code with all features working
.jpg / .png	Screenshot showing the title screen, gameplay, and leaderboard
.mp4	Video of a complete playthrough — one win and one loss
.pdf	1-page report: how linked lists, file I/O, and pass-by-reference were used

Grading Criteria:

- **Correct use of linked lists** (30%)
- **Proper memory management** (20%)
- **Collision detection accuracy** (20%)
- **Keyboard input handling** (15%)
- **Game logic and win/lose conditions** (15%)

Bonus points for smooth gameplay, creative enhancements, and code organization.

Deadline: Friday, May 15, 2026

Good luck, and have fun coding! 🚀